Language Models and Agents — Individual Project

Phase 3: Fine-Tuning, Comparative Reasoning Evaluation, and Final Cross-Lingual Report

A from-scratch decoder-only Transformer, pretrained and fine-tuned independently on Hindi (Model H, high-resource) and Nepali (Model L, low-resource)

Satyajit Priyadarshi | Roll No. 2025201055
M.Tech Computer Science, IIIT Hyderabad | Language Models and Agents, Monsoon 2026

Checkpoints: https://www.kaggle.com/datasets/satyagudu/hindi-finetuned-model

https://www.kaggle.com/datasets/satyagudu/neplai-finetuned-model

Executive Summary

This report consolidates all three phases — **Phase 1** (Data & Tokenization), **Phase 2** (Pretraining & Generation), and **Phase 3** (Fine-Tuning & Comparative Reasoning) — into one comparative evaluation of a decoder-only Transformer trained independently on **Hindi (Model H, higher-resource)** and **Nepali (Model L, lower-resource)**. Both models share an identical architecture, 24,918,528-parameter budget, and training recipe; only language, corpus, and SentencePiece tokenizer differ. Phase 1/2/3 training standardized on a **32,000**-token vocabulary (a 50k vocabulary was the Phase-1-only deliverable choice, reported here for completeness). The goal: test how far the Hindi/Nepali resource gap persists once both models are pushed from raw next-token prediction into structured, multi-entity comparative reasoning.

Metric / Stage	Model H (Hindi)	Model L (Nepali)	Winner
Phase 1 – BPE Fertility (32k)	1.249	1.298	H (more compact subwords)
Phase 1 – Test UNK Rate	0.253%	0.431%	H (lower UNK)
Phase 2 – Val. Loss (best)	4.0032	4.3677	H (lower per-token loss)
Phase 2 – Perplexity	54.77	78.85	H (more predictable tokens)
Phase 2 – Bits-per-Byte	0.558	0.476	L (better byte compression)
Phase 3 – Val. Peak Acc.	60.50%	62.17%	L (+1.67 pts)
Phase 3 – Test Exact Match	59.23% (1777/3000)	56.53% (1696/3000)	H (+2.70 pts)

Table 1: Key results at a glance.

In short: Hindi is easier to model at the token level and generalises better on harder 3-/4-way reasoning, while Nepali — once its critical tokenizer bug (§3.2) was fixed — matches Hindi almost exactly on direct lookup and edges it out on peak validation accuracy and 2-way comparison. The resource gap is real but far narrower on reasoning than intrinsic LM metrics alone predict.

1 Phase 1 Recap: Data & Tokenizer Construction

Two fully independent pipelines (no document sharing, no shared vocabulary) were built. **Hindi** drew on national newspapers, literature archives (Kavita Kosh, Gadya Kosh, Rekhta), Wikipedia/Wikisource, and AI4Bharat Sangraha. **Nepali** drew on an active news ecosystem (Ekantipur, Onlinekhabar, Setopati, Ratopati, Nagarik News) plus Sangraha. Original asyncio crawlers (robots.txt-respecting, rate-limited, Devanagari-ratio filtered $\geq$0.55, NFC-normalized, hash-deduplicated) fed both corpora, split 98/1/1 (train/val/test) by a seeded RNG (SEED=42).

Language	Manual (Crawl) Tokens	Public (Sangraha)	Combined Total	Manual Frac.
Nepali (L)	∼99.07M (99,896 docs)	∼400.00M	∼499.07M	∼19.85%
Hindi (H)	∼98.82M (93,784 docs)	∼400.00M	∼498.82M	∼19.81%

Table 2: Final merged corpus statistics – token scale is matched across languages.

Four BPE vocabulary sizes [16k,32k,50k,64k] were swept per language; the smallest vocabulary within 3% fertility tolerance of the best candidate was selected (Phase 1 deliverable: 50k). Phase 2/3 training instead standardized on **32k** for both languages to keep embedding-table size (and total parameters) identical.

Metric (32k, held-out TEST)	Hindi (H)	Nepali (L)
BPE Fertility	1.249	1.298
Avg. Chars/Token	4.08	4.97
UNK Rate	0.253%	0.431%
Unique Tokens Observed	47,714	49,748

Table 3: Held-out test tokenizer performance at the training-time 32k vocabulary.

Hindi has lower fertility, UNK rate, and chars/token at *every* vocab size tested – consistent with Hindi's larger, more homogeneous corpus giving BPE more repeated sub-word patterns to merge into longer tokens.

Vocab Size (Nepali)	Fertility	UNK Rate	Chars/Token	Status
16,000	1.415	0.410%	4.56	Rejected
32,000	**1.298**	**0.447%**	**4.97**	**Used for Phase 2/3 training**
50,000	1.246	0.466%	5.18	Phase 1 deliverable choice
64,000	1.224	0.474%	5.28	Rejected
Vocab Size (Hindi)	**Fertility**	**UNK Rate**	**Chars/Token**	**Status**
16,000	1.329	0.198%	3.84	Rejected
32,000	**1.249**	**0.211%**	**4.08**	**Used for Phase 2/3 training**
50,000	1.215	0.217%	4.20	Phase 1 deliverable choice
64,000	1.200	0.220%	4.25	Rejected

Table 4: Tokenizer candidate vocabulary sweep (validation split), Nepali (top) and Hindi (bottom). The smallest vocabulary within 3% fertility tolerance of the best candidate was the Phase 1 selection rule.

2 Phase 2 Recap: Pretraining Architecture & Evaluation

A decoder-only (GPT-style) Transformer was implemented from PyTorch primitives only (`nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, `nn.Dropout` – no built-in attention/Transformer modules) and pretrained independently per language. **Shared architecture (both models, 32k vocab):** 24,918,528 total params (24,689,152 non-embedding); 6 layers; $d_{model} = 448$; 8 heads ($d_{head} = 56$); FFN width 1024; context 512; learned absolute positional embeddings; tied embeddings (32,000×448); AdamW (β_1=0.9, β_2=0.95, wd=0.1); cosine LR, 300 warmup steps, peak lr 3×10^{-4}; effective batch 64. This "shallow-and-wide" recipe (Pathway B, 6 layers) was chosen over a deep/narrow 14-layer alternative (Pathway A) after a Hindi pilot: Pathway A reached val_loss 4.6932 (PPL 109.20) vs. Pathway B's 4.3992 (PPL 81.39) with fewer parameters; the winning recipe was reused unchanged for Nepali.

Architecture Pathway (Hindi pilot)	Layers	Params	Val. Loss / PPL
Pathway A – deep & narrow	14	23.9M	4.6932 / 109.20
Pathway B – shallow & wide (chosen)	**6**	**24.9M**	**4.3992 / 81.39**

Table 5: Architecture pathway comparison run on Hindi. Pathway B was reused unchanged for Nepali.

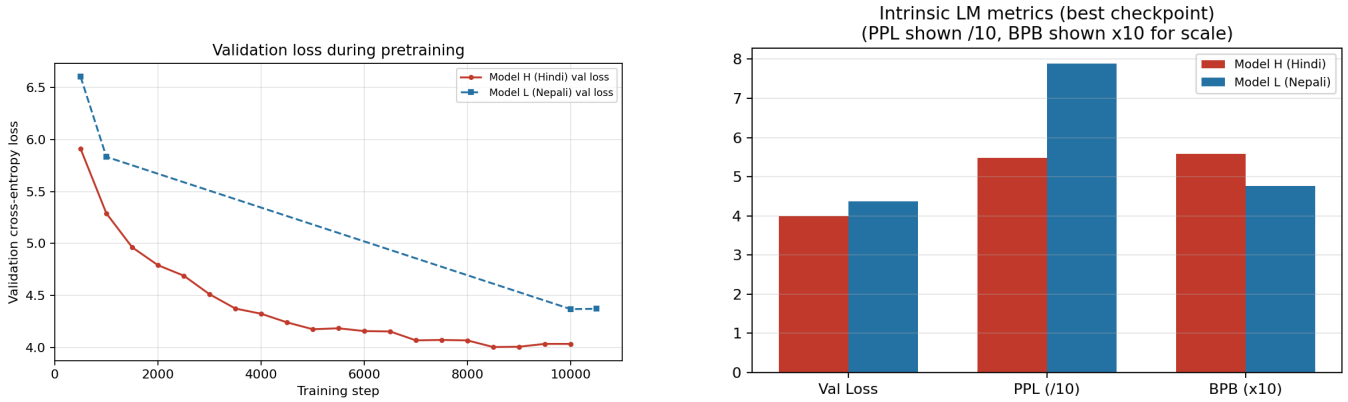


Figure 1: Left: validation cross-entropy loss vs. step, Model H vs. Model L. Right: best intrinsic metrics side-by-side (PPL/10, BPB×10, shared axis).

Metric	Model H (Hindi)	Model L (Nepali)
Val. cross-entropy (best)	4.0032	4.3677
Perplexity	54.77	≈78.85
Bits-per-byte (BPB)	0.558	0.476

Table 6: Pretraining intrinsic metrics.

Nepali reaches higher loss/PPL despite identical architecture and near-identical token budget – plausibly because its smaller, less diverse, morphologically richer corpus makes next-token prediction intrinsically harder. Yet Nepali's BPB is *lower* (0.476 vs 0.558): its tokenizer packs more bytes per token, so higher per-token loss does not mean worse byte-level compression – BPB shows the languages are closer in true modelling difficulty than PPL suggests.

Generation (50 held-out prefixes; greedy, temperature 0.5/1.0/1.5, top-k 20/50, top-p 0.8/0.9/0.95, beam-4): Greedy decoding degenerates into loops for both languages (repetition rate 0.60–0.76). Top-k sampling is the most reliable strategy for both (Hindi top-k=50: chrF 18.31; Nepali top-k=50: BLEU-4 2.21, chrF 22.89), keeping repetition low. Beam search collapses into repetition for both (0.83 Hindi / 0.59 Nepali) – its mode-seeking objective reproduces greedy-style loops even with a wider search.

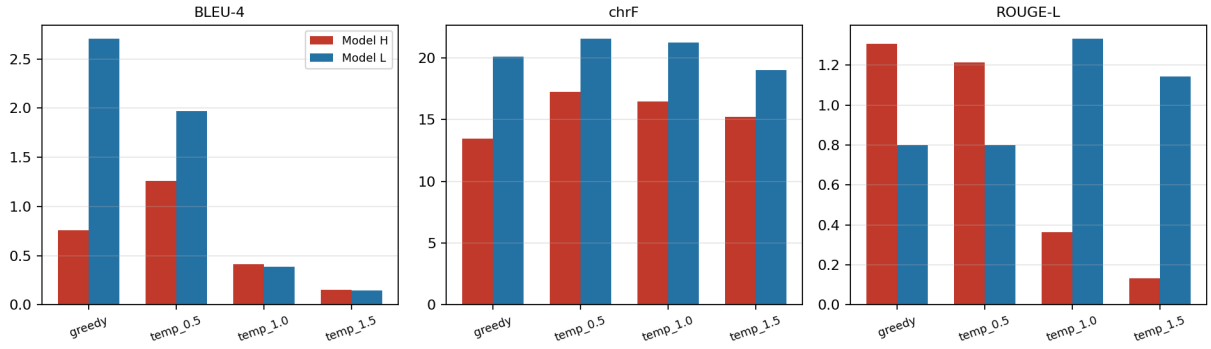


Figure 2: BLEU-4, chrF and ROUGE-L across decoding settings, Model H vs. Model L.

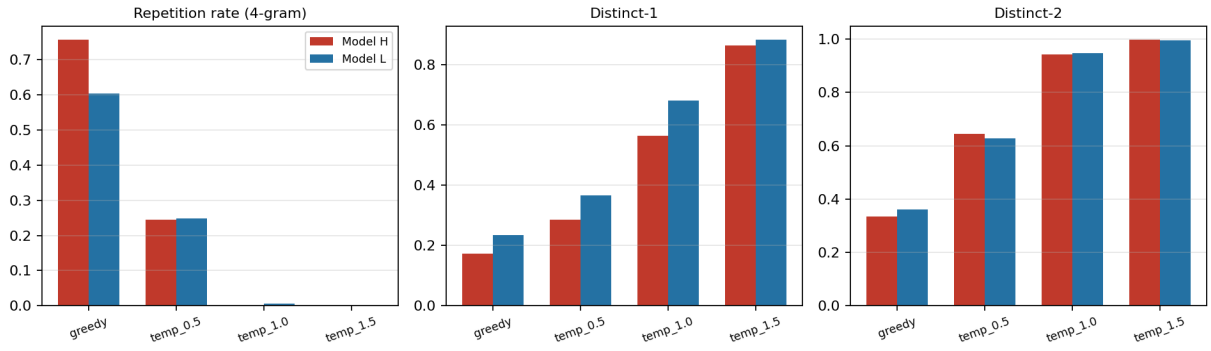


Figure 3: Repetition rate (4-gram) and Distinct-1/Distinct-2 diversity across decoding settings, Model H vs. Model L.

Attention (pretrained checkpoints): Both models show the same depth-wise structure: layers 0–1 are high-entropy/moderately local (context-building), layers 3–4 specialise into narrow, near-deterministic local/positional heads (Model H layer-4 head-4 entropy 0.296; Model L layer-4 head-7 entropy 0.261), and layer 5 re-aggregates via long-range, content-based heads. This similarity despite unrelated corpora/vocabularies suggests depth-wise attention structure is largely a function of the shared architecture/recipe, not the language.

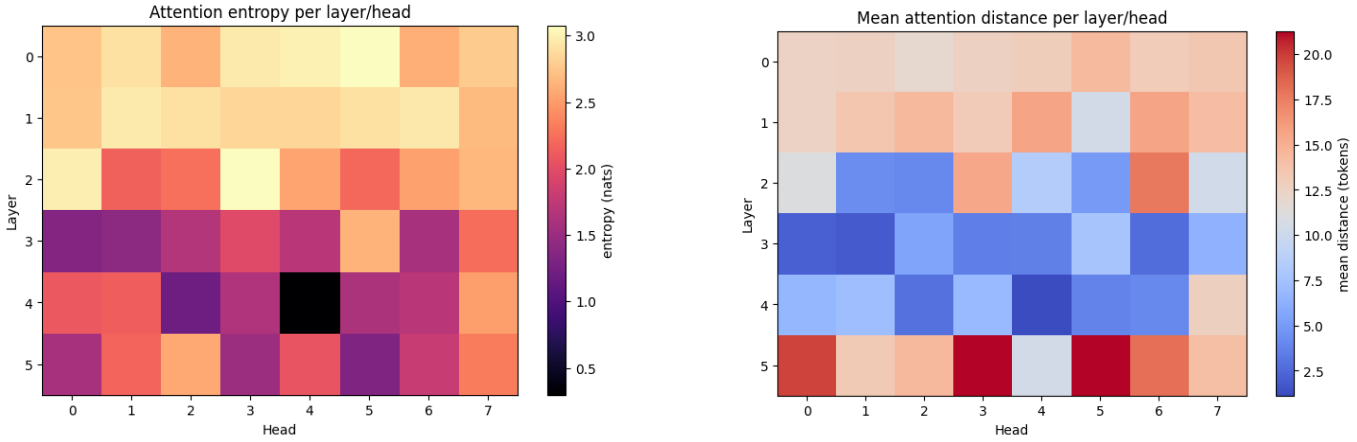


Figure 4: Model H (Hindi) pretrained-checkpoint attention entropy (left) and mean distance (right), per layer/head.

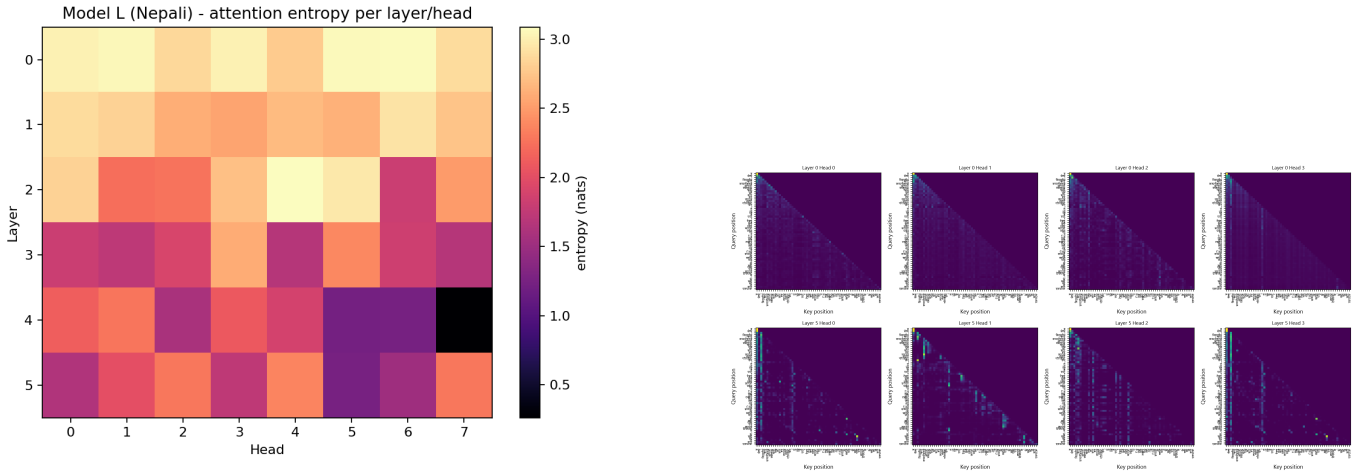


Figure 5: Model L (Nepali) pretrained-checkpoint attention entropy (left) and mean distance (right), per layer/head – the same broad structural pattern as Model H (Fig. 2), despite an unrelated corpus and vocabulary.

3 Phase 3: Synthetic Reasoning Data & Fine-Tuning Method

Phase 3 moves both checkpoints into structured, multi-entity comparative reasoning to test whether the Phase 1/2 resource-tier gap persists, narrows, or reverses after identical-schema fine-tuning.

Task schema (five types, shared quotas both languages): `bind_entity_to_value` (direct lookup), `bind_value_to_entity` (reverse lookup), `compare_2way`, `compare_3way`, `compare_4way` – e.g. दीपकका अंक = ७० | सुरेशका अंक = ७७ | सबैभन्दा बढी अंक कसको छ? → सुरेश.

3.2 The ASCII-numeral tokenizer bug (Model L). Initial Nepali fine-tuning crashed to 2.16% accuracy on numeric binding tasks: ASCII digits ('0'–'9') mapped to token ID 0 (`<unk>`) under `nepali_bpe_32000.model`, collapsing all numeric values to `<unk>` and vanishing the corresponding attention value representations. **Fix:** dataset generation was converted to Devanagari numerals (०१२३४५६७८९), which map to clean single BPE tokens – this jumped Soft validation accuracy from 58.50% → 98.33% and enabled the 62.17% Hard validation peak. This is the single largest tokenizer-driven factor in the whole Phase 3 result set, and is specific to the lower-resource tier.

Sequential fine-tuning (Soft → Hard): Pretrained (32k) → Soft (30k/3k/3k) → `soft_best.pt` (98.33% val acc.) → Hard (30k/3k/3k, position-balanced, equality task dropped) → `hard_best.pt` (62.17% val acc., Nepali). Both languages use identical hyperparameters: lr 1×10^{-5}, 3 epochs (938 steps/epoch, batch 32), first-token loss weight 3.0 (weights the answer-carrying token 3×), warmup 0.06, weight decay 0.01. Soft and Hard generators were diffed line-for-line across languages (identical quotas, split sizes, position-balancing, zero train/val/test overlap) – the only differences are surface language and (Nepali only) the numeral fix.

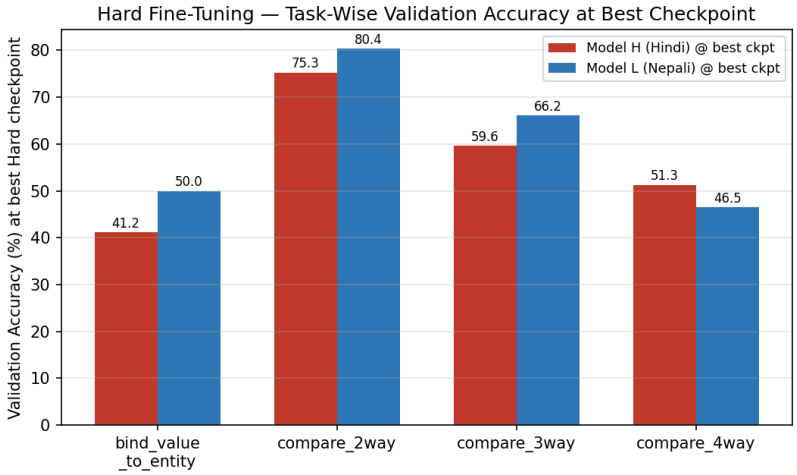


Figure 6: Task-wise validation accuracy at each language's best Hard checkpoint (Hindi: step 2,400; Nepali: step 2,700).

Both languages converge on Soft within 3 epochs to near-identical accuracy (Hindi 98.17% vs. Nepali 98.33% at step 2,400); Nepali overtakes Hindi from step 900 on. On Hard, Hindi's checkpoint peaks and plateaus one eval earlier (60.50% at steps 2,400 and 2,700) while Nepali keeps improving to 62.17% at step 2,700. Nepali starts far behind on `bind_value_to_entity` at step 300 (8.82% vs. Hindi's 38.24%, a direct echo of the numeral bug still being resolved) but closes most of the gap by the final checkpoint.

Model H (Hindi) Soft checkpoint – full held-out TEST evaluation. Unlike the Nepali soft run (which only logged the training-time reasoning-accuracy proxy), the Hindi soft checkpoint was additionally scored on the complete, held-out 3,000-example Soft TEST split (overall 97.40%, 2922/3000) – a clean read on how well Soft-stage fine-tuning alone generalizes before Hard fine-tuning ever begins.

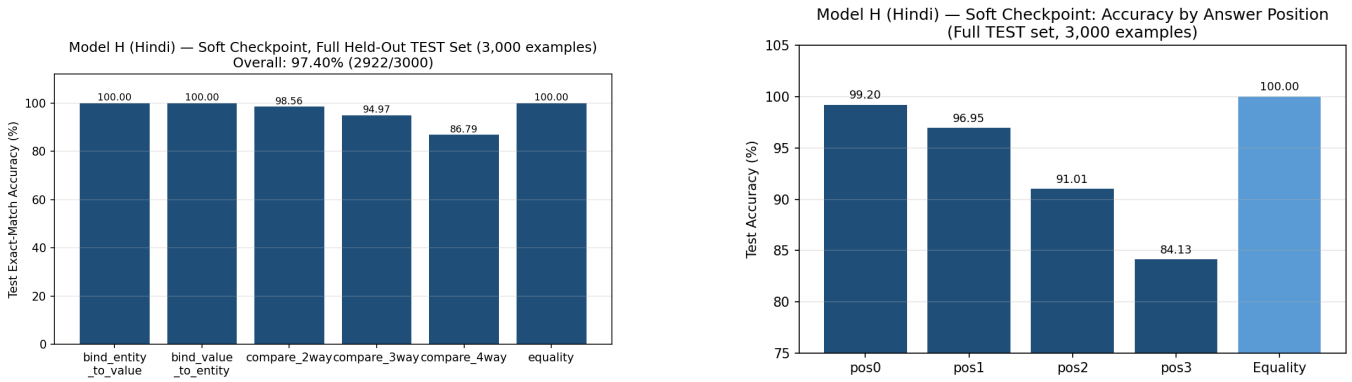


Figure 7: Model H (Hindi) Soft checkpoint, held-out TEST set: accuracy by task (left, overall 97.40%) and by answer position (right, equality shown separately).

Direct binding and equality are perfectly solved (100.00%) and 2-way comparison is nearly perfect (98.56%), but accuracy still degrades with both entity count (99.48%→94.97%→86.79% for 2/3/4 entities) and answer position (99.20%→96.95%→91.01%→84.13% for pos0–pos3) – the same position-bias signature documented for the Hard-fine-tuned Nepali model (§4), now shown to already be present at the Soft stage, before any Hard-curriculum training, and for the higher-resource language too.

Task Type (Soft, 30k train)	Train Quota	Train Share
bind_entity_to_value	5,998	20.0%
bind_value_to_entity	4,492	15.0%
compare_2way	7,546	25.2%
compare_3way	5,982	19.9%
compare_4way	3,005	10.0%
equality	2,977	9.9%

Table 7: Soft dataset – shared task quotas (both languages, 36,000 total → 30k/3k/3k). The equality task (e.g. two entities sharing the same value) is dropped for the harder curriculum below.

Task Type (Hard, 30k train)	Example Count	Share	Position Balance
bind_entity_to_value	1,800	5.0%	n/a (single entity)
bind_value_to_entity	1,800	5.0%	n/a (single entity)
compare_2way	5,400	15.0%	pos0=2,700 / pos1=2,700
compare_3way	12,600	35.0%	pos0=pos1=pos2=4,200
compare_4way	14,400	40.0%	pos0=pos1=pos2=pos3=3,600

Table 8: Hard dataset – shared task quotas & position balance (both languages, 36,000 total). Both languages' generators report identical zero-overlap dedup guarantees (Train∩Val = Train∩Test = Val∩Test = 0) and the same five randomized surface-form templates per language.

Pipeline Stage / Metric	Model H (Hindi)	Model L (Nepali)	Note
Soft best – training reasoning acc.	98.17% (step 2,400)	98.33% (step 2,400)	Within 0.2 pts
Soft best – full TEST exact match	97.40% (2922/3000)	not separately logged	Hindi-only full test
Hard best – overall val. acc.	60.50% (step 2,400)	62.17% (step 2,700)	Nepali overtakes late
Hard test – bind_entity_to_value	100.00% (139/139)	100.00% (139/139)	Tied
Hard test – bind_value_to_entity	42.36% (61/144)	42.36% (61/144)	Tied (reverse-lookup ceiling)
Hard test – compare_2way	73.95% (369/499)	76.35% (381/499)	Nepali +2.4 pts
Hard test – compare_3way	62.79% (670/1067)	56.98% (608/1067)	Hindi +5.8 pts
Hard test – compare_4way	46.74% (538/1151)	44.05% (507/1151)	Hindi +2.7 pts
Hard test – overall exact match	59.23%	56.53%	Hindi +2.7 pts

Table 9: Model H vs. Model L – final checkpoint snapshot across the whole Soft → Hard pipeline. With identical architecture, hyperparameters, and dataset schema, the two languages land within a few points of each other at every stage.

4 Final Evaluation & Comprehensive Results

Raw pretrained base models score 0.00% on the 3,000-example held-out Hard test set because they emit free-running continuations rather than the required single-token structured answer – fine-tuning teaches the output format itself before reasoning accuracy is even measurable.

Evaluation Metric	Model H Pretrained	Model H Hard Fine	Model L Pretrained	Model L Hard Fine
Validation Peak Accuracy	0.00%	60.50%	0.00%	62.17%
Test Exact Match Accuracy	0.00%	59.23% (1777/3000)	0.00%	56.53% (1696/3000)
First-Token Accuracy	0.00%	59.23% (1777/3000)	0.00%	56.53% (1696/3000)

Table 10: Pretrained vs. Hard fine-tuned accuracy, evaluated on `hard_test.jsonl` (3,000 held-out prompts).

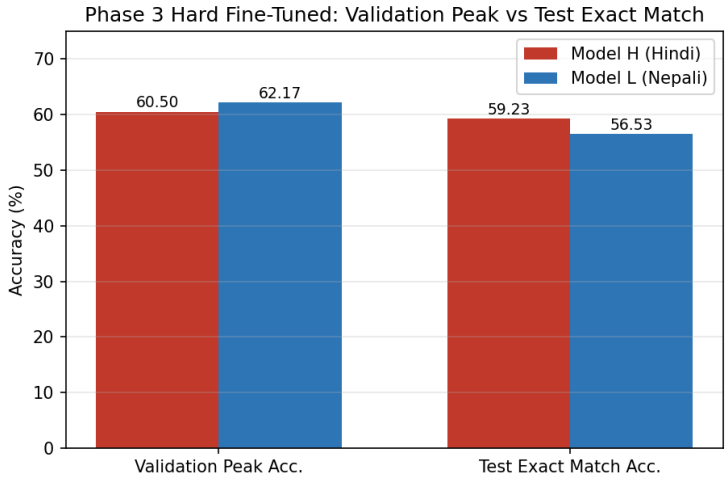


Figure 8: Validation peak vs. test exact-match accuracy, Hard fine-tuned models, Model H vs. Model L.

Task Type	Model H Test Acc.	Model L Test Acc.	Leader
bind_entity_to_value (direct)	100.00% (139/139)	100.00% (139/139)	Tied
bind_value_to_entity (reverse)	42.36% (61/144)	42.36% (61/144)	Tied
compare_2way	73.95% (369/499)	76.35% (381/499)	L (+2.40)
compare_3way	62.79% (670/1067)	56.98% (608/1067)	H (+5.81)
compare_4way	46.74% (538/1151)	44.05% (507/1151)	H (+2.69)
Overall exact match	**59.23%**	**56.53%**	H (+2.70)

Table 11: Task-wise & overall Hard-test exact-match accuracy, Hard-fine-tuned models (32k vocab).

Direct binding is tied at 100%; the two diverge as entity count scales: L holds a small edge at 2-way, H pulls progressively ahead at 3-/4-way – Hindi's advantage is concentrated in harder, higher-entity-count reasoning, not in easier binding tasks.

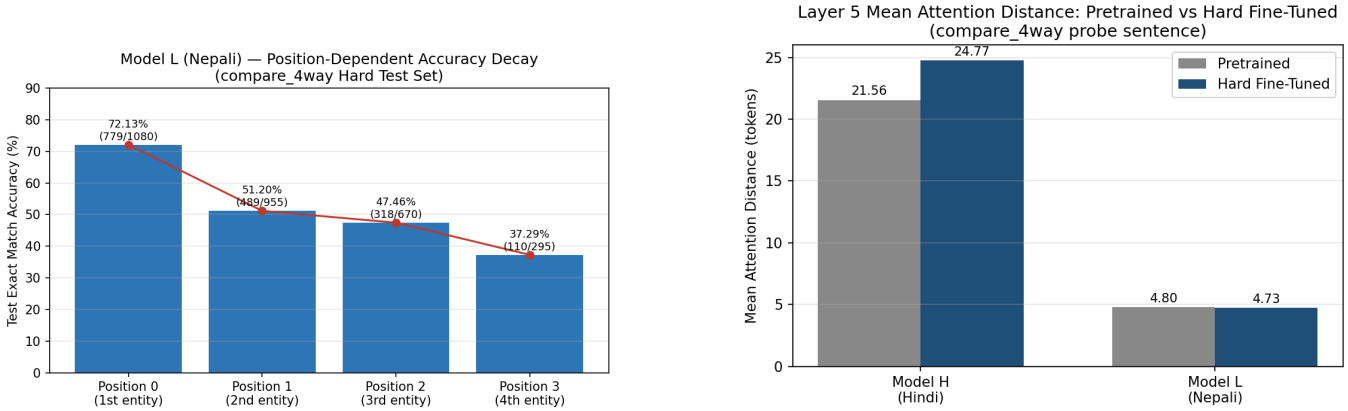


Figure 9: Left: Model L (Nepali) `compare_4way` accuracy vs. gold-entity position in prompt. Right: layer-5 mean attention distance on a `compare_4way` probe, pretrained vs. hard-fine-tuned, both languages.

Position bias (Nepali, compare_4way): accuracy decays monotonically with the target entity's position: pos0 72.13% → pos1 51.20% → pos2 47.46% → pos3 37.29% – nearly halving from first- to last-mentioned entity.

Attention diagnostics (layer 5, entropy/distance/concentration, Pretrained→Soft→Hard): Model H shows large, task-scaled shifts – distance grows with fine-tuning on every task (`compare_4way`: 21.56→24.77 tokens) and concentration drops sharply (`compare_4way`: 0.397→0.243), i.e. attention becomes more far-reaching and distributed as task difficulty rises. **Model L** shows almost no movement across any stage/task (distance stays in a narrow 4.7–7.0 token band; `compare_4way` concentration 0.2364→0.2352→0.2387) even though task accuracy rises from 0% to 44–100% – Nepali appears to solve reasoning through a different, more locally-scoped attention strategy than Hindi.

5 Core Research Questions

Q1 – Data scale & quality. Scale is matched (∼500M tokens each, ∼100M crawled + ∼400M Sangraha), so raw token count is not a confound. The asymmetry is *quality*: Hindi text is standardized across dozens of major portals/archives/Wikipedia, giving low subword fragmentation; Nepali is concentrated across fewer portals with higher morphological inflection, spelling variance, and English code-mixing, pushing BPE toward more, shorter pieces. This quality gap – not scale – drives Nepali's higher fertility (1.298 vs 1.249) and UNK rate (0.431% vs 0.253%), which is the root cause of the numeral tokenizer bug (§3.2).

Q2 – LM vs. reasoning across tiers. Hindi is unambiguously easier to model per-token: lower fertility/UNK, lower val. loss (4.0032 vs 4.3677), lower PPL (54.77 vs 78.85); Nepali's lower BPB (0.476 vs 0.558) shows part of this is a tokenizer-fertility artifact rather than pure modeling difficulty. After fine-tuning, the gap narrows sharply and partly reverses: L wins peak val. accuracy (62.17% vs 60.50%) and 2-way comparison (76.35% vs 73.95%); H retains the edge on 3-way (62.79% vs 56.98%), 4-way (46.74% vs 44.05%) and test exact match (59.23% vs 56.53%); direct binding is tied at 100%. **Takeaway:** intrinsic LM quality is not a reliable predictor of downstream structured-reasoning accuracy at this scale – a ∼24-point PPL gap collapses to a ∼1–3 point reasoning gap after identical-schema fine-tuning.

Q3 – Tokenizer/corpus factors hurting the lower-resource model.

1. **ASCII-numeral OOV bug (critical):** ASCII digits mapped to `<unk>` under the Nepali tokenizer, crashing binding accuracy to 2.16% pre-fix; switching to Devanagari numerals restored 98.33% Soft / 62.17% Hard peak accuracy. The single largest tokenizer-driven factor overall, and unique to the low-resource tier.

2. **Morphological subword fragmentation:** higher fertility (1.298 vs 1.249) and longer chars/token (4.97 vs 4.08) mean multi-entity prompts tokenize longer for Nepali, increasing attention-memory load exactly on the 3-/4-way tasks where L underperforms.
3. **Source concentration:** Nepali's crawl draws on fewer independent portals, plausibly reducing lexical/register diversity and contributing to its higher Phase 2 validation loss/PPL.

Q4 – Evidence for the observed differences. *Attention-routing:* fine-tuning converts Hindi's late-layer heads into long-range key-retrieval mechanisms (4-way mean distance 21.56→24.77 tokens, pretrained→hard); Nepali's corresponding shift is far smaller (4.80→4.73 tokens), consistent with its weaker gain on exactly the tasks needing long-range routing. *Position bias:* the 72.13%→37.29% position-decay (§4) is a second, independent line of evidence – causal self-attention retrieves earlier-mentioned entities more easily, and this compounds with entity count. *Architectural similarity:* despite unrelated corpora/vocabularies, both models specialise attention at the same depths (local/positional at layers 3–4, long-range aggregation at layer 5), indicating depth-wise structure comes from the shared architecture/recipe, while the H-vs-L gap comes from data-quality and tokenizer factors layered on top.

6 Qualitative Successes & Failure Modes

Direct binding (100%): दीपकका अंक = ७० → दीपकका अंक कति हुन्? → ७० [correct, both languages]. **2-way (74–76%):** दीपकका अंक = ७०, सुरेशका अंक = ७७ → सबैभन्दा बढी अंक कसको छ? → सुरेश [correct]. **3-way (57–63%):** सञ्जय: ३५, राहुल: ३७, सुरेश: ३२ → सबैभन्दा बढी मूल्य कसको छ? → राहुल [correct]. **Failure A – multi-entity attention decay (4-way, 44–47%):** with 4 entities the model consistently biases toward later/more-recent-looking entities instead of the true max/min (small 24.9M-parameter models struggle to hold four key-value pairs in working memory and rank them correctly). **Failure B – causal masking asymmetry (reverse lookup, 42.36%, tied):** autoregressive attention favours the forward direction (entity→value) it was trained on most; reverse (value→entity) lookup forces backward retrieval and both languages fail it identically – an architectural/objective limitation, not a resource-tier effect.

7 Bonus (Optional): Ablation – No Positional Embeddings (Hindi)

As an optional bonus, Model H was retrained with positional embeddings fully removed (`use_pos_emb=False`, same 6-layer/24.9M-param architecture), and the full Phase 2 evaluation suite (loss/PPL/BPB, 10 decoding strategies, attention diagnostics) was rerun.

Metric	Standard H (w/ pos.)	Ablated H (no pos.)	Impact
Val. cross-entropy loss	4.0032	4.3214	+0.3182 loss penalty
Perplexity (PPL)	54.77	75.29	+20.52 higher PPL
Bits-per-byte (BPB)	0.558	0.602	+0.044 BPB penalty

Table 12: Intrinsic LM metrics: standard vs. no-positional-embedding Model H.

Decoding	BLEU-4	chrF	ROUGE-L	Rep. Rate	Distinct-1	Distinct-2
Greedy	0.795	14.391	1.382	0.5476	0.1797	0.4001
Temp 0.5	0.769	15.655	2.267	0.1925	0.2873	0.6494
Temp 1.0	0.634	16.147	0.689	0.0000	0.5836	0.9567
Temp 1.5	0.117	15.679	0.000	0.0000	0.8518	0.9947
Top-k 20	0.604	17.029	1.013	0.0394	0.3671	0.8243
Top-k 50	0.687	16.982	0.536	0.0060	0.4236	0.8992
Top-p 0.8	0.595	16.449	0.727	0.0000	0.4591	0.9182
Top-p 0.9	0.310	16.235	0.167	0.0180	0.5153	0.9206
Top-p 0.95	0.395	16.125	0.000	0.0012	0.5345	0.9430
Beam-4	1.382	15.175	1.467	0.6814	0.1822	0.3598

Table 13: Ablated-H generation quality across the full 10-strategy decoding sweep.

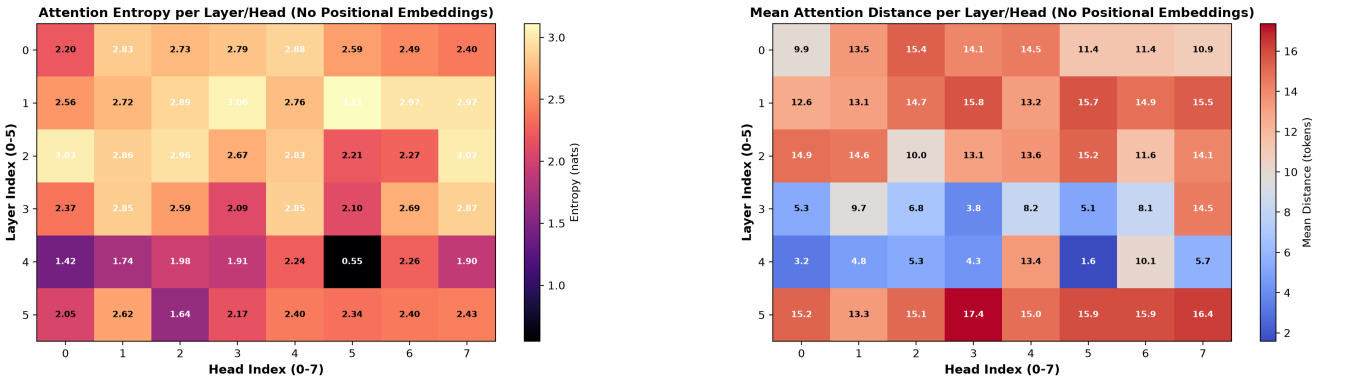


Figure 10: Ablated Model H (no positional embeddings): layer×head attention entropy (left) and mean distance (right), 6 layers × 8 heads.

Layer-wise summary: mean entropy/distance per layer – L0: 2.616 nats / 12.63 tok; L1: 2.881 / 14.44; L2: 2.731 / 13.40; L3: 2.551 / 7.69; L4: 1.748 / 6.05; L5: 2.255 / 15.54. Most-focused head: L4H5 (entropy 0.548, distance 1.58 tok). Most long-range head: L5H3 (entropy 2.166, distance 17.37 tok).

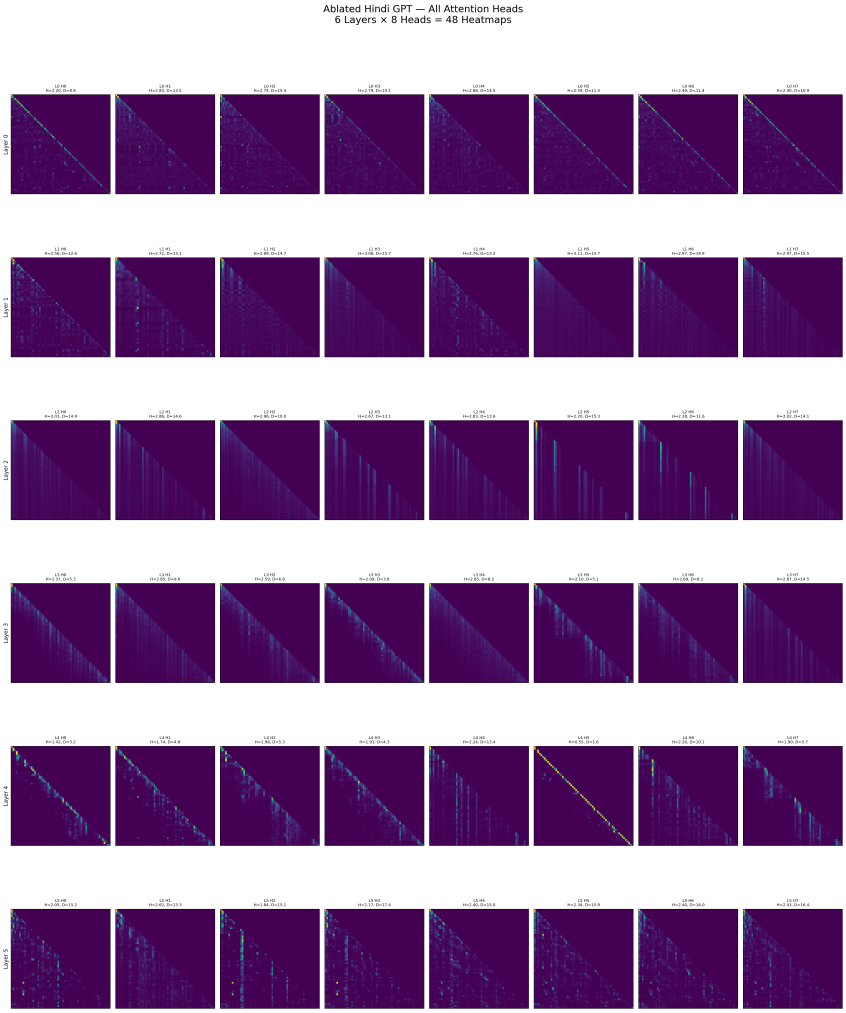


Figure 11: Ablated Model H – full 48-head attention heatmap grid (6 layers × 8 heads, no positional embeddings). Without a positional signal, early/late layers spread attention broadly and diagonally-local heads seen in the standard model (Fig. 1–type patterns) are far less consistent.

What breaks without position information:

1. **Permutation invariance:** with no positional signal, query-key dot products carry no index information – swapping token positions yields identical pre-mask representation sets.

2. **Locality collapse via causal-mask over-reliance:** the causal mask becomes the only remaining asymmetric signal; early-layer local/diagonal focus is lost (layer-0 mean distance jumps from 4.82 to 12.63 tokens vs. the standard model).
3. **Loop degeneracy:** deterministic decoding degrades sharply (greedy repetition rate 54.76%, beam-4 68.14%) – without position, the model has a harder time tracking "where" it is in the generated sequence, worsening the standard model's own repetition failure mode.

Overall, removing positional embeddings costs ∼20.5 PPL points and 0.044 BPB, and specifically degrades the model's ability to build the local, position-anchored attention heads (layers 3–4) that the standard model relies on for both fluent generation and, by extension, the position-sensitive comparative-reasoning tasks in Phase 3.

Implementation & Evaluation Protocol Notes

Evaluation protocol. `final_eval_hard.py` reports first-token accuracy (does the top-1 next token match the gold answer's first token) and exact-match accuracy (greedy decode, ≤12 new tokens, stop at EOS); the two are numerically identical in every logged run, confirming a correct first token reliably implies a fully correct short answer at this task's 1–3-token answer length. The first-token loss weight of 3.0 (§3.3) directly targets this metric.

Leakage controls. All dataset generators MD5-hash every prompt and reject duplicates before quota assignment, then re-verify zero train/val/test overlap (confirmed 0/0/0 in both languages' logs). A per-task, per-answer-position quota (max_per_position = quota ÷ num_entities) enforces the balanced position distribution used in §4/§3. Comparison values are drawn from a narrow, close-together range (a random start in [10,90] plus small offsets ≤12) specifically to block the model from solving comparisons via superficial digit-length or magnitude shortcuts.

Attention capture vs. production forward pass. Two parallel implementations of the same architecture exist: `final_eval_hard.py` (and `hard_after_soft.py`/`soft_finetuned.py`) use PyTorch's fused `F.scaled_dot_product_attention` for speed during training/evaluation, while `attention_analysis.py` reimplements attention manually (explicit $QK^\top$, softmax, masking) purely so the full (B, nhead, T, T) probability tensor can be captured for every forward pass. Both share identical weights and produce identical logits – the manual version is used only for the post-hoc attention analysis throughout this report, never for training.